Spring 2026
Data Structures Section D20 — Lab 04

Topic

Abstract Data Types and Applications: 1. Stack 2. Queue

Objective

- Understand Abstract Data Types: Stack & Queue
- Understand the use of Global Functions on Stack & Queue
- Applications of Stack & Queue

Outcomes

1. Implement Functions on Stack & Queue
2. Write Complete C++ Programs for Problems using Stack & Queue only

Content

The following sections will be covered during this lab session.

1. **Basic Functions** using Queue & Stack
2. **Complete Programs** for Given Problems using Queue & Stack only

Tasks

Students are required to complete the following tasks in lab timings.

For the tasks 1-5, create a private repository in your account on Github: **S26_DSA_D11_Lab_4.** Upload codes with the names **Task_1**, **Task_2** and so on it.

Mandatory for Each Task

The following Steps must be **included** as **commented text** in **each .cpp file of each task**, before function main. **Missing these steps mean that you have NOT done the work yourself and it is from ChatGPT. (Please use Plain English or Urdu and No C++ Coding terms)**

Please note that these steps are more important than writing code, even at the expanse of code incompletion.

1. **Clearly and in details**, explain what you understand about the task at hand after carefully reading the given description and code.
2. Describe how will you do the task and the **solution** e.g. how to code a given functions or solution to the problem. You should use detailed steps and use the notations given in the slides for the Queue e.g. (A → B → C →) to show the progression of the solution.
3. Describe what **functions** you will **reuse** and **why**. Furthermore, explain the solution in terms of functions used. (**make sure your code is as per your solutions steps). Show** Queue e.g. (A → B → C →) to show the progression of the queue after each function call.
4. Describe **your learning** from the task

5. Describe the **skills** you have developed and what type of problems you can code based on your experience of doing the task

Class Code Files to be Used in the Tasks

Must use the Class Code File as is. In case you need to change then provide reasons in the comments.

These File are tested on Visual Studio 2022 to Compile and Run Correctly

```
// Stack.h
class Stack
{
 public:
  Stack(int _maxCapacity = 0);
  Stack(const Stack & AA);
  virtual void push(int v) = 0; // increments currentIndex and add value if not full
  virtual bool pop(int & value) = 0; // if not empty returns the value at the currentIndex and decrements
currentIndex
  bool isEmpty( );
  bool isFull( );
 protected:
  int currentIndex;
  int * values;
  int maxCapacity;
};

 Stack::Stack(int _maxCapacity)
 {
   currentIndex = -1;
   maxCapacity = _maxCapacity;
   values = new int[maxCapacity]{0};
 }
 Stack::Stack(const Stack & s)
 {
   currentIndex = -1;
   maxCapacity = s.maxCapacity;
   values = new int[maxCapacity]{0};
   for(int i = 0; i < maxCapacity; i++)
     {
       values[i] = s.values[i];
     }
 }

bool Stack::isEmpty( )
{
  return (currentIndex < 0);
}
bool Stack::isFull( )
  {
  return (currentIndex == maxCapacity - 1);
}
```

```
// MyStack.h
```

```
class MyStack: public Stack
{
 public:
  MyStack(int _currentIndex = 0): Stack(_currentIndex){};
  MyStack(const MyStack & ma): Stack(ma){};
  void push(int v);
  bool pop(int & v);
};

void MyStack::push(int v)
{
  if(!isFull())
    {
      values[++currentIndex] = v;
      return;
    }
     cout << "Array is full\n";
}

bool MyStack::pop(int & v)
  {
    if(!isEmpty())
    {
      v = values[currentIndex--];
      return true;
    }
    return false;
}
```

```
// Queue.h
class Queue
{
 public:
  Queue(int cap) : startOfQIndex(-1), topOfQIndex(-1), maxCapacity(cap), count(0) {}
 Queue(const Queue & q) : startOfQIndex(q.startOfQIndex), topOfQIndex(q.topOfQIndex),
maxCapacity(q.maxCapacity), count(q.count){}
  virtual ~Queue() {}

  bool isEmpty() const;
  bool isFull() const;
  virtual void enqueue(const  int & value) = 0;
  virtual int dequeue() = 0;
  virtual void display(ostream &) const = 0;
protected:
    int startOfQIndex;     // corresponds to front
    int topOfQIndex;       // corresponds to rear
    int maxCapacity;       // total size of array
    int count; // keep a count of values in queue
};

  // Check if queue is empty
bool Queue::isEmpty() const
```

```
{
     return (count == 0);
}
// Check if queue is full
bool Queue::isFull() const
{
  return (count == maxCapacity );
}
```

```
// MyQueue.h
class MyQueue: public Queue
{
public:
  MyQueue(int s): Queue(s)
  {
    if(s != 0)
      {
            values = new int[maxCapacity]{0};
      }
  }
  MyQueue(const MyQueue & mq): Queue(mq)
  {
    if(maxCapacity != 0 || values == nullptr)
      {
            values = new int[maxCapacity]{0};
            for(int i = 0; i < count; i++)
              {
                values[i] = mq.values[i];
              }
      }
  }

  ~MyQueue()
  {
    if(values != nullptr)
      {
            delete [] values;
            values = nullptr;
      }
  }
  void enqueue(const int & value);
```

Task 1

Implement the code for the following functions, use only **Queue (Stack may be used but provide a sound explanation in the comments of why that particular function can't be implemented with Queue only)** and no other data type **(except int, float, char and string)**. Test your code after adding function codes to the main.cpp (**must not change unless you can point out errors and describe in comments)** below and upload it to your repository**.** The name of the code on your repository should be **Task_1.cpp.** Include **Stack.h, MyStach.h, Queue.h and MyQueue.h** in **Task_1.cpp**

```
// main.cpp for task 1
#include <iostream>
using namespace std;

#include <iostream>
using namespace std;
#include "Stack.h"
#include "MyStack.h"
#include "Queue.h"
#include "MyQueue.h"

void read(Queue *, istream &); // read from user or file until queue is full
void display(Queue *, ostream &); // Display the values in the Queue
void append(Queue * src, Queue * dst); // append src to dst
void append(Queue * src, Queue * dst, int no); // append no of values from
src to dst
void reverseAppend(Queue * src, Queue * dst);// append src in dst in reverse
order
int getSize(Queue * q);
void stats(Queue * q, int & sum, float & average);
bool isEqual(Queue *, Queue * );

const int MAX_SIZE = 5;
int main()
{
  Queue* A = new MyQueue(MAX_SIZE);
  int n = 0;
  read(A, cin);
  cout << " Queue A Values: ";
  display(A, cout);
  Queue* B = new MyQueue(MAX_SIZE);
  append(A, B);
  cout << " Queue B Values: ";
  display(B, cout);
  Queue* C = new MyQueue(MAX_SIZE);
  C->enqueue(9);
  C->enqueue (7);
  C->enqueue(5);
  append(A, C, 2);
  cout << "Queue C Values after append of 2 values from A: ";
  display(C, cout);
  Queue * D = new MyQueue(MAX_SIZE);
  reverseAppend(C, D);
  cout << "Queue D Values after reverse append from V: ";
  display(D, cout);
  int mx = 0;
  float ag = 0.0;
```

```
  stats(A, mx, ag);
  cout << "Queue A Values: ";
  display(A, cout);
  cout << "Average is " << ag << " maximum is " << mx << endl;
  Queue* M = new MyQueue(MAX_SIZE);
  Queue* N = new MyQueue(MAX_SIZE);
  Queue* P = new MyQueue(MAX_SIZE);
  append(A, M);
  append(N, M);
  append(N, P);
  P->enqueue(111);
  if(!isEqual(A, M))
    {
      cout << "Not Equal:\n";
    }
  else
    {
      cout << "Equal:\n";
    }

  if(!isEqual(A, P))
    {
      cout << "Not Equal:\n";
    }
  else
    {
      cout << "Equal:\n";
    }
  return 0;
}
```

Task 2

Implement the code for the following functions, use only **Queue (Stack may be used but provide a sound explanations in the comments of why that particular function can't be implemented with Queue only)** and no other data type **(except int, float, char and string)**. Test your code after adding function codes to the main.cpp (**must not change unless you can point out errors and describe in comments)** below and upload it to your repository**.** The name of the code on your repository should be **Task_2.cpp.** Include **Stack.h, MyStach.h, Queue.h and MyQueue.h** in (**reuse functions from task 1**)

```
// main.cpp for Task 2
#include <iostream>
using namespace std;

#include "Stack.h"
#include "MyStack.h"
#include "Queue.h"
#include "MyQueue.h"

void display(Queue *, ostream &); // Display the values in the Queue
void append(Queue * src, Queue * dst); // append src to dst
void append(Queue * src, Queue * dst, int no); // append no of values from
src to dst
```

```
void reverseAppend(Queue * src, Queue * dst);// append src in dst in reverse
order
int getSize(Queue * q);
void insert(Queue * src, Queue * dst, int pos);
void insert(Queue * q, int v, int pos); // insert a value in the queue at
the given position
void shiftRight(Queue * q, int pos);
void shiftLeft(Queue * q, int pos);
bool isEqual(Queue *, Queue * );

const int MAX_SIZE = 10;
int main()
{
  Queue* A = new MyQueue(MAX_SIZE);
  A->enqueue(10);
  A->enqueue(9);
  A->enqueue(2);
  A->enqueue(5);
  A->enqueue(-7);
  cout << " Queue A Values: ";
  display(A, cout);
  Queue* B = new MyQueue(MAX_SIZE);
  insert(B, -100, 2);
  cout << " Queue B Values after insert: ";
  display(B, cout);
  insert(A, B, 2);
  cout << " Queue B Values after insert: ";
  display(B, cout);
  shiftRight(B, 2);
  cout << " Queue B Values after shiftRight from pos 2";
  display(B, cout);
  shiftLeft(B, 2);
  cout << " Queue B Values after shiftLeft from pos 2: ";
  display(B, cout);

return 0;
}
```

Task 3

Implement the code for the following functions, use only **Queue (Stack may be used but provide a sound explanations in the comments of why that particular function can't be implemented with Queue only)** and no other data type **(except int, float, char and string)**. Test your code after adding function codes to the main.cpp (**must not change unless you can point out errors and describe in comments)** below and upload it to your repository**.** The name of the code on your repository should be **Task_3.cpp.** Include **Stack.h, MyStach.h, Queue.h and MyQueue.h** in (**reuse functions from task 2**)

```
// main.cpp for Task 2
#include <iostream>
using namespace std;

#include "Stack.h"
#include "MyStack.h"
#include "Queue.h"
#include "MyQueue.h"

void display(Queue *, ostream &); // Display the values in the Queue
void append(Queue * src, Queue * dst); // append src to dst
void append(Queue * src, Queue * dst, int no); // append no of values from
src to dst
void reverseAppend(Queue * src, Queue * dst);// append src in dst in reverse
order
int getSize(Queue * q);
void insert(Queue * src, Queue * dst, int pos);
void insert(Queue * q, int v, int pos); // insert a value in the queue at
the given position
void shiftRight(Queue * q, int pos);
void shiftLeft(Queue * q, int pos);
bool isEqual(Queue *, Queue * );
int get(Queue * q, int pos); // get from the queue the value at the given
position after removing from the Queue
void sort(Queue *); // Sort the values in the queue in ascending order
bool isPalindrome(Queue *); // check if the values in the queue are
Palindrome
void isPalindrome(Queue *, bool & ); //where single digits are stored and
you can't use any other Queue or Stack or ADT
void rotateLeft (Queue * q, int pos);
void rotateRight (Queue * q, int pos);

const int MAX_SIZE = 10;
int main()
{
  Queue* A = new MyQueue(MAX_SIZE);
  A->enqueue(10);
  A->enqueue(9);
  A->enqueue(2);
  A->enqueue(5);
  A->enqueue(-7);
  cout << " Queue A Values: ";
  display(A, cout);

  rotateRight(A, 2);
  cout << " Queue A Values after rotateRight from pos 2";
  display(A, cout);
```

```
  rotateLeft(A, 2);
  cout << " Queue B Values after rotateLeft from pos 2: ";
  display(A, cout);
  int v = get(A, 3);
  cout << "Value at pos 3: " << v << endl;
  cout << " Queue A Values after get from pos 3: ";
  display(B, cout);
  if(isPalindrome(A))
    {
      cout << "A is a Palindrome\n";
    }
  else
    {
      cout << "A is NOT a Palindrome\n";
    }
  Queue* C = new MyQueue(MAX_SIZE);
  C->enqueue(234);
  C->enqueue(56);
  C->enqueue(786);
  C->enqueue(56);
  C->enqueue(234);
  if(isPalindrome(C))
    {
      cout << "C is a Palindrome\n";
    }
  else
    {
      cout << "C is NOT a Palindrome\n";
    }
  Queue* D = new MyQueue(MAX_SIZE);
  C->enqueue(3);
  C->enqueue(5);
  C->enqueue(7);
  C->enqueue(5);
  C->enqueue(3);
  bool isIt = false;
  isPalindrome(D, isIt);
  if(B)
    {
      cout << "D is a Palindrome\n";
    }
  else
    {
      cout << "D is NOT a Palindrome\n";
    }
  sort(B);
  cout << " Queue B Values after sort in ascending \n";
  display(B, cout);
  return 0;
}
```

Task 4

Implement complete C++ program for the following Problem and upload it to your repository**. You can only use Queue and no other Data Types (except int, float, char and string) (use of Stack is allowed if justification is provided). Must reuse function codes from previous Tasks** The name of the code on

your repository should be **Task_4.cpp.** Include Stack.h, MyStach.h, Queue.h and MyQueue.h in **Task_4.cpp**

Problem Code using Queue & Stack (DS Midterm Q2)

Write a function **reverseFirstKElements(queue, k)** in the ***main file*** that reverses the order of the first **k** elements in a Queue, leaving the remaining elements in the same relative order. You are not allowed to implement the Queue class or its functions; the following are provided and work correctly:

enqueue(value); **dequeue()** (*returns first value*); **isEmpty()** and **isFull()** (*both return bool*)

Note: You are not given a **size**() function. If required, you must determine the total number of elements in **queue** by using only the available queue functions. Further, you are not allowed to use any data structure other than queue and stack. Use as many queue or stack objects as required. Assume stack functions detailed in question 1 (**push**, **pop**, **isFull**, **isEmpty**) are also available for use. Also assume **k** is valid and less than or equal to the total number of elements.

Example 1:

k = 3; **queue:** [10, 20, 30, 40, 50]
Output: [30, 20, 10, 40, 50]

Example 2:

k = 4; **queue:** [15, 0, -3, 99]
Output: [99, -3, 0, 15]

Task 5

Implement complete C++ program for the following Problem and upload it to your repository**. You can only use Queue and no other Data Types (except int, float, char and string) (use of Stack is allowed if justification is provided).** You may create a Class Order and make the Queue of Orders. **Must reuse function codes from previous Tasks** The name of the code on your repository should be **Task_5.cpp.** Include **Stack.h, MyStach.h, Queue.h and MyQueue.h** in **Task_5.cpp**

ADT Queue Application Example

In a fast food restaurant, customer place an order to the waiter, the waiter places the order to the cook. The cook takes each order prepares the food, give it to the waiter and removes the order, the waiter serves the food to the customer. Important thing is to make sure the food is served to the customer in the same sequence it is ordered. Please implement this whole mechanism using a Queue ADT.

Your program should keep track of the sequence of when the order is placed and when the order is served. Furthermore, calculate the amount of money earned. Example orders: Burger, Sandwich, French Fries, Fried Chicken and Pizza.

Test on different Order Sequences

And display price and time on each

Order placement and fulfillment

Item	Price	Time Taken in Minutes
Burger	200	20
Sandwich	250	30
French Fries	100	10
Fried Chickens	400	40
Pizza	350	40